

PART 2: CODE REVIEW SAMPLE

Below is a small full-stack feature from a habit-tracking product: a Node.js API and the React dashboard component that consumes it. Treat it as a colleague's pull request. Review both files as you would in a real team: identify the issues, explain why they matter, prioritise them, and suggest fixes. Submit your review as written comments in a markdown file.

Backend: `server.ts`

```
// Habit dashboard API (excerpt)
import express from "express";
import mysql from "mysql2/promise";

const app = express();
app.use(express.json());

const db = mysql.createPool({
  host: "prod-db.internal",
  user: "root",
  password: "Passw0rd123!",
  database: "healthapp",
});

// Dashboard data: user's habits with their last 7 days of logs
app.get("/api/dashboard", async (req, res) => {
  const userId = req.query.userId as string;
  const [habits]: any = await db.query(
    `SELECT * FROM habits WHERE user_id = ${userId}`
  );

  const result: any[] = [];
  habits.forEach(async (habit: any) => {
    const [logs]: any = await db.query(
      `SELECT * FROM habit_logs WHERE habit_id = ${habit.id} AND log_date > DATE_SUB(NOW(), INTERVAL 7 DAY)`
    );
    result.push({ ...habit, logs });
  });

  res.json(result);
});

// Log a habit for today
app.post("/api/logs", async (req, res) => {
  const { userId, habitId, value, email } = req.body;
  console.log(`User ${email} logged habit ${habitId}: ${value}`);

  const [existing]: any = await db.query(
    `SELECT id FROM habit_logs WHERE user_id = ${userId} AND habit_id = ${habitId} AND log_date = CURDATE()`
  );

  if (existing.length === 0) {
    db.query(
      `INSERT INTO habit_logs (user_id, habit_id, value, log_date) VALUES (${userId}, ${habitId}, '${value}', CURDATE())`
    );
  }
  res.json({ success: true });
});
```

```

    } else {
      res.json({ success: false, reason: "already logged today" });
    }
  });

  app.use((err: any, req: any, res: any, next: any) => {
    res.status(500).json({ error: err.stack });
  });

  app.listen(3000);

```

Frontend: HabitDashboard.tsx

```

import { useEffect, useState } from "react";

interface Habit {
  id: number;
  name: string;
  target: number;
  logs: { value: string; log_date: string }[];
}

export default function HabitDashboard({ userId }: { userId: number }) {
  const [habits, setHabits] = useState<Habit[]>([]);
  const [search, setSearch] = useState("");
  const [filtered, setFiltered] = useState<Habit[]>([]);

  useEffect(() => {
    fetch(`/api/dashboard?userId=${userId}`)
      .then((res) => res.json())
      .then((data) => setHabits(data));
  });

  useEffect(() => {
    setFiltered(habits.filter((h) => h.name.includes(search)));
  }, [search]);

  const completionRate = (habit: Habit) => {
    const days: string[] = [];
    for (let i = 0; i < 7; i++) {
      const d = new Date();
      d.setDate(d.getDate() - i);
      days.push(d.toISOString().slice(0, 10));
    }
    const completedDays = days.filter((day) =>
      habit.logs.some((log) => log.log_date.startsWith(day))
    );
    return Math.round((completedDays.length / 7) * 100);
  };

  const logHabit = (habitId: number) => {
    const updated = habits;
    updated.find((h) => h.id === habitId)!.logs.push({
      value: "done",
      log_date: new Date().toISOString(),
    });
    setHabits(updated);
  };

```

```
    fetch("/api/logs", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ userId, habitId, value: "done" }),
    });
  };

  return (
    <div>
      <input
        placeholder="Search habits"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
      {filtered.map((habit, index) => (
        <div key={index}>
          <h3>{habit.name}</h3>
          <span>{completionRate(habit)}% complete</span>
          <button onClick={() => logHabit(habit.id)}>Log today</button>
        </div>
      ))}
    </div>
  );
}
```